

Spark vs. Ray: The Data Engineering Duel

DA3408 Assignment 3 | M. Sai Teja, Roll No. DA24B031 | NYC Yellow Taxi trips, Jan–Jun 2023 (19.5M input rows) | 5 October 2026

1. Summary

- **Winner for this pipeline and hardware: Apache Spark.** It finished the six-month pipeline in **186.6 s** and **253.2 s** (two runs); Ray took **780.7 s** (one run), so Ray was **3.1x to 4.2x slower**, with higher peak memory (5.8 vs 6.7 GiB cluster working set).
- **The outputs are identical:** 18,253,326 rows, the same 30 columns and dtypes, an identical order-independent row hash, and 0 duplicate rows in each.
- **Python UDF:** Ray's Python-native path cost **31.7 s** of overhead against Spark's **50.2 s** (1.6x less). Spark's built-in expression was still faster than Ray's vectorized one (3.7 s vs 5.7 s).
- **Scope:** six of the nine downloaded months. Ray could not complete nine months inside this VM's memory and disk (Section 7). All numbers come from my own runs, saved as JSON artifacts and screenshots.

2. Cluster setup and evidence

	Spark (standalone)	Ray (Ray Data)
Version	Spark 3.5.7 (Python 3.11.17)	Ray 2.49.2 (Python 3.11.17, pyarrow 19.0.1)
Topology	1 master + 2 workers (2 cores, 2 GiB executor each) + 1 driver container (768 MB)	1 head (no compute CPUs) + 2 workers (2 CPUs each, 1 GiB object store each)
Compute	4 executor cores, 4 GiB executor memory	4 CPUs, ~2.1 GiB total object store
Host	One 6-core, 7.2 GiB aarch64 Ubuntu VM (UTM/QEMU). Every node is a Docker container (3 GiB limit per worker) on one bridge network.	Same VM; the two clusters ran one at a time (not enough RAM for both).

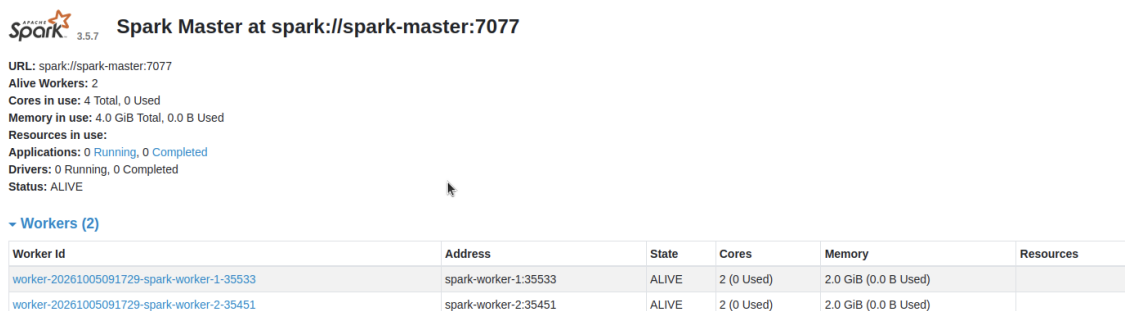


Figure 1a. Spark Master UI (port 8080): 2 alive workers, 4 cores, 4.0 GiB.

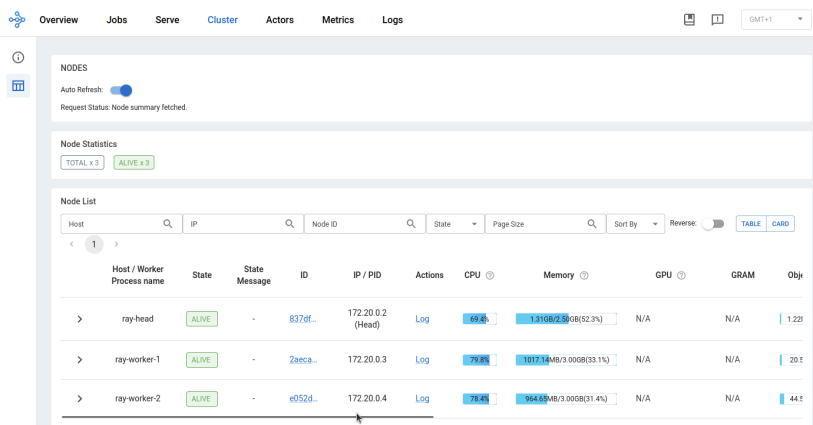


Figure 1b. Ray Dashboard (port 8265): 3 alive nodes (head + 2 workers), captured under load.

Deviation to note: the workers are containers on one VM, not separate machines, so network and disk are shared and the resource numbers reflect a single 7.2 GiB host.

3. Pipeline and parity

Both scripts (`spark_clean.py`, `ray_clean.py`) implement the same rules, taken from the settings recorded in each run's metrics file. Timing covers reading through the finished Parquet export in both; cluster start-up and output inspection are excluded.

Step	Rule applied identically in Spark and Ray
Ingestion	Six monthly Parquet files plus the taxi-zone lookup from a shared directory. Schemas normalized to 19 source columns (months differ in physical types).
Cleansing	Drop rows with a null in any of the 19 columns; keep $1 \leq \text{passengers} \leq 8$, $0 < \text{distance} \leq 1000$, $0 < \text{duration} \leq 24$ h, $\text{fares} \geq 0$, $\text{VendorID/RatecodeID} > 0$, flag in {Y,N}; pickup in Jan–Jun 2023; global DISTINCT across all files; timestamps kept as local wall-clock microseconds.
Transformation	Two INNER joins to the zone lookup (pickup and dropoff; blank labels become Unknown), then a shared Python function computes three features (avg_speed_mph, fare_per_mile, fare_per_minute, rounded to 6 decimals) plus pickup hour and trip duration.
Export	Snappy Parquet, 64 files, 30 columns. Input 19,493,620 rows, output 18,253,326 rows (6.4% removed).

Parity check (6 months)	Spark	Ray	Result
Rows	18,253,326	18,253,326	match
Distinct rows (duplicates)	18,253,326 (0)	18,253,326 (0)	no duplicates in either
Columns and dtypes	30	30	match
Order-independent row hash (all columns)	168357332003157618401821125	168357332003157618401821125	identical
1-month full diff (EXCEPT ALL both ways)	2,884,428 rows	2,884,428 rows	0 / 0 differences

The six-month check is a row-hash comparison, not a row-by-row diff: a bucketed row-by-row verifier matched all 16 of the 128 buckets it completed (0 differences) before I stopped it for time. The one-month output was verified row by row (Figure 10).

Evidence: Figures 5 and 10 (screenshots/4_parity_duplicates_udf_terminal.png).

4. Performance (Jan–Jun 2023, Spark two runs, Ray one run)

Metric	Spark run 1	Spark run 2	Ray	Ray / Spark
Execution time, ingestion to export	186.6 s	253.2 s	780.7 s	4.2x / 3.1x
Peak CPU, cluster (400% = 4 cores)	487%	437%	503%	similar
Peak memory, cluster (working set)	5.83 GiB	5.81 GiB	6.67 GiB	1.14x - 1.15x
Output on disk (64 files)	695 MB	not read back	937 MB	1.35x

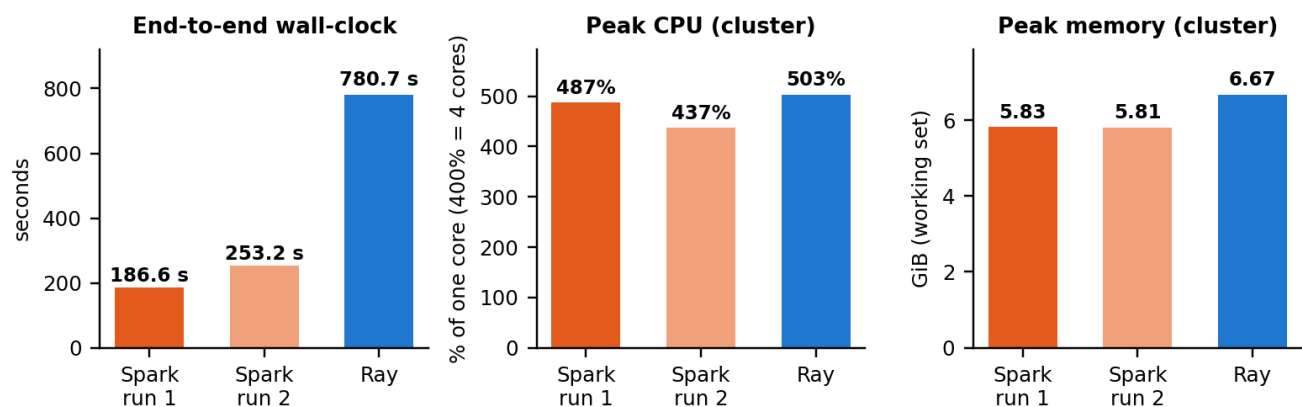


Figure 2. End-to-end time and cluster peaks (sampled from container metrics by the measurement wrapper). Spark’s two runs differ by 36%, so the gap to Ray is a range, not a point. Memory is working set; the separate “charged” memory was ~6.9 GiB for both and appears to include page cache, so I do not use it to compare. Evidence: Figures 6 to 10.

Run 2 was read back for its headline values only. Parity was verified on run 1’s output; run 2 wrote the same 18,253,326 rows.

Why Ray was slower here

Both frameworks kept all four cores near saturation, so the gap is not idle CPU. The evidence points at Ray’s data movement under a small memory budget (a diagnosis from the run logs, not a profile):

- **Super-linear scaling.** Ray ran January alone (2.88M output rows) in 24.6 s, but six months (18.25M rows, 6.3x more) in 778.6 s: **31.6x** the time.
- **Global DISTINCT is a sort.** Ray Data has no hash distinct, so my implementation range-partitions and sorts all rows on every column, an all-to-all barrier whose intermediate data (2.7 GB or more) is larger than the ~2.1 GiB combined object store. Spark’s distinct is a hash aggregation with managed spill.
- **The failures agree:** the nine-month Ray attempts died of an out-of-memory kill and “local disk is full”, and an earlier six-month variant lost objects and stalled (Section 7).
- Ray’s output is also larger on disk (937 vs 695 MB). The row hash shows the data is identical, so this is a Parquet encoding difference, not a data difference.

5. Python UDF deep-dive

To isolate the UDF I read the finished output (18.25M rows) in each framework and ran the *same* per-row Python function (speed = distance / hours, from the pickup and dropoff timestamps) three ways: a baseline scan of the three columns, the Python UDF, and a native version. A warm-up read precedes timing. Spark writes to a no-op sink; Ray materializes its blocks.

Measurement (seconds)	Spark	Ray	Reading
Baseline scan (3 columns)	0.98	2.74	different sinks, so only roughly comparable
Python UDF, total	51.14	34.46	Ray runs 1–3: 34.46 / 34.90 / 48.95 (median 34.90)
Native / vectorized, total	3.73	5.68	Spark JVM expression vs Ray numpy
UDF overhead (UDF total - baseline)	50.15	31.72	Ray 1.6x lower
Native overhead	2.75	2.94	~same
UDF total / native total	13.7x	6.1x	penalty for leaving the native engine

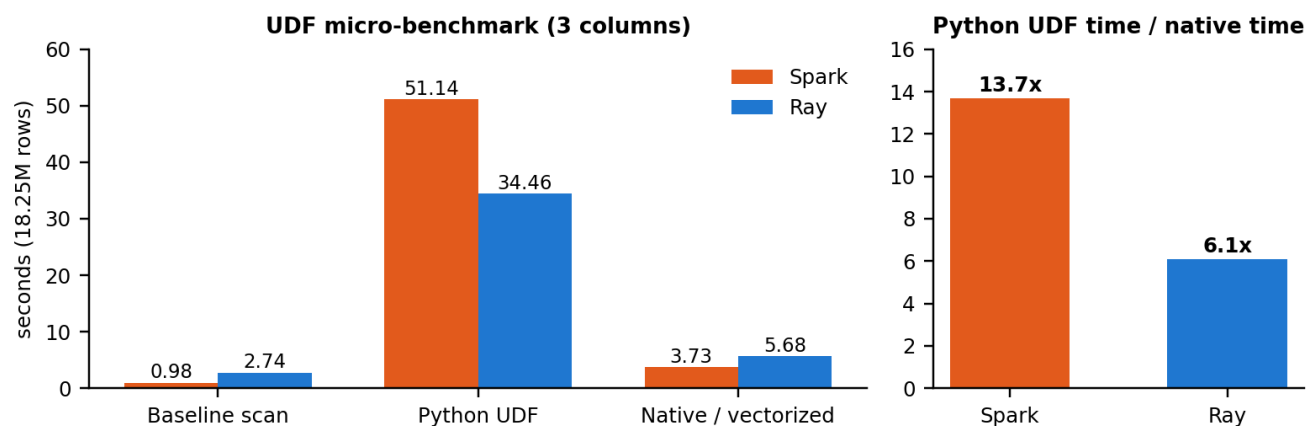


Figure 3. UDF micro-benchmark on the 18.25M-row Spark output. Evidence: Figures 4 and 5.

What this shows. Spark's Python UDF is 13.7x slower than its own native expression, because every row crosses the JVM–Python boundary (serialization in both directions). Ray's batches already live in Python, so the penalty is smaller (6.1x) and its UDF overhead is 18.4 s lower. That is the Python-native advantage. It is also small: 18.4 s is about 3% of the 527 to 594 s end-to-end gap, so the UDF does not explain why Ray lost; the relational stages do.

Caveats. Spark's UDF figure is one run (Ray's is three, with run 3 about 40% slower, likely shared-VM noise). Spark passes `datetime` objects and Ray passes `pandas.Timestamp` objects to the same function. Spark's native expression truncates to whole seconds while the UDF keeps microseconds. An earlier Ray run without a warm-up (baseline 7.07 s, UDF 29.99 s) is excluded because its cold-cache baseline made the native overhead negative.

6. Conclusion: which framework for which project

	Choose Spark when (BI-first)	Choose Ray when (AI-first)
Workload	Relational ETL: joins, global distinct, aggregations, SQL, scheduled reporting	Python-heavy feature engineering, ML preprocessing that feeds training, GPU or actor-based pipelines
Evidence here	3.1x to 4.2x faster end to end, lower memory, no tuning crisis; native expressions are cheapest	Python UDF overhead 1.6x lower, data stays in pandas/Arrow in the same runtime as the model code
Cost	Python UDFs are slow (13.7x native); avoid them or use vectorized pandas UDFs	Relational operators (distinct, shuffles) need hand-tuning and plenty of object-store memory

Verdict: for this preprocessing pipeline on this hardware, **Spark wins**. For a new AI-first project I would still pick Ray for the Python-native model-facing stages, but let Spark (or a hash-based engine) do the heavy relational cleaning, and give Ray a larger object store than the ~2.1 GiB available here.

7. Performance Tuning Note (AI use and what the real runs showed)

LLM assistants (ChatGPT and Claude) helped translate the pipeline between PySpark and Ray Data, suggested the Ray tuning below, and wrote the benchmark scripts. I set up and configured both clusters, ran every benchmark, and took every screenshot myself; no performance number was generated by an AI. Each AI suggestion was tested on the VM:

AI-suggested Ray change	Real result on this VM
Sort-based global distinct, 256 range buckets, one read operator for all files	Kept. Correct (0 duplicates, hash parity).
Pull-based sort shuffle instead of the default push-based shuffle	1 month: 20.1 s vs 62.0 s. Kept.
Nine months, one read operator per file plus union	No result; stopped after ~20 min.
Nine months, single bounded-batch reader	Container out-of-memory kill (exit 137).
Nine months, 64 MiB blocks	OutOfDiskError during the sort reduce.
Six months, two separate Ray joins to the zone table	Lost-object recovery, then failure after ~16 min.
Six months, one native join on a combined pickup/dropoff lookup (70,225 pairs, equivalent to both joins)	Succeeded: 780.7 s, exact parity. Used for the reported number.

Limits. One VM (6 cores, 7.2 GiB) hosts every node; six months, not nine; end-to-end: two Spark runs (186.6 s and 253.2 s) and one Ray run; UDF benchmark: Ray three runs, Spark one; the six-month parity check is a row hash. Absolute times would differ on real multi-machine clusters.

Evidence screenshots (also submitted as separate image files)

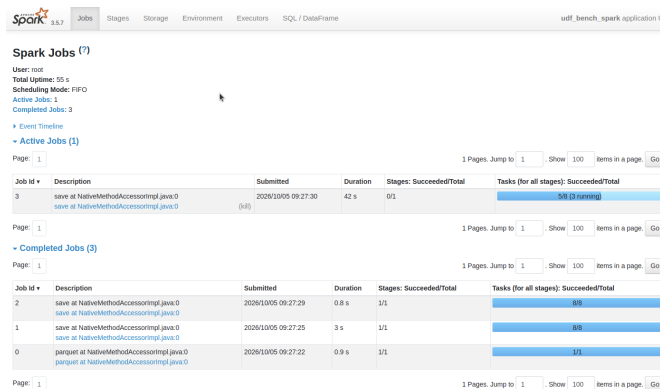


Figure 4. Spark UI (port 4040) during the Spark UDF benchmark on the six-month output: three jobs completed, the Python-UDF job running (42 s so far). From screenshots/3_spark_ui_4040_udf_benchmark.png.

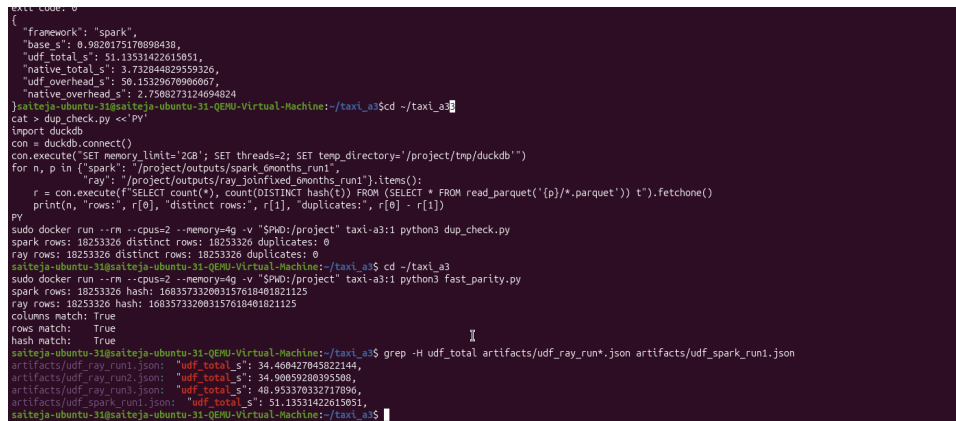


Figure 5. Terminal: Spark UDF benchmark result (exit code 0), duplicate check (0 duplicates in both outputs), row-hash parity (all True) and the four UDF totals. From screenshots/4_parity_duplicates_udf_terminal.png.

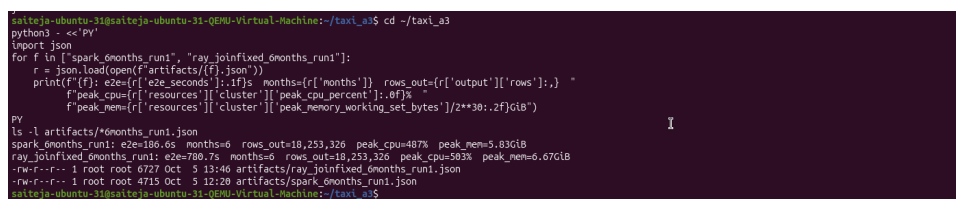


Figure 6. Terminal: headline results read from the saved run files. Spark 186.6 s and Ray 780.7 s, both 6 months and 18,253,326 output rows; peak CPU and working-set memory match Figure 2. From screenshots/6_headline_results_spark_vs_ray.png.

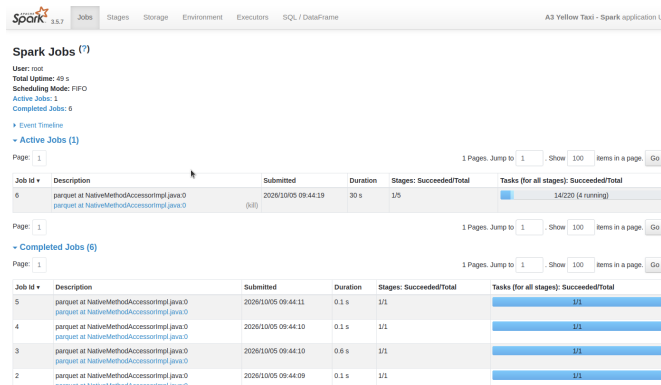


Figure 7. Spark UI (port 4040) during the main pipeline run (run 2): application “A3 Yellow Taxi - Spark”, job 6 at 14/220 tasks with 4 running, matching the 4 executor cores.

```
saiteja-ubuntu-31@saiteja-ubuntu-31-QEMU-Virtual-Machine:~/taxi_a3$ cd ~/taxi_a3
curl -fsS http://127.0.0.1:8080/json/ | grep -o 'aliveworkers' | wc -l # expect 2
sudo sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
sudo python3 measure_run.py \
  --metrics artifacts/spark_6months_run2.json \
  --containers spark-master spark-worker-1 spark-worker-2 spark-driver \
  --workers spark-worker-1 spark-worker-2 \
  --docker exec spark-driver spark-submit \
  --master spark://spark-master:7077 --deploy-mode client \
  --driver-memory 768m --executor-memory 2g --executor-cores 2 --total-executor-cores 4 \
  --conf spark.driver.host=spark-driver --conf spark.driver.bindAddress=0.0.0.0 \
  --conf spark.driver.port=7079 --conf spark.blockManager.port=7080 \
  spark_clean.py --months 6 --partitions 64 \
  --output /project/outputs/spark_6months_run2 \
  --metrics /project/artifacts/spark_6months_run2.json
"aliveworkers" : 2
3
```

Figure 8. Run 2 command: spark_clean.py --months 6 submitted to the standalone master with 2 alive workers, Spark 3.5.7.

```
26/10/05 09:44:02 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20261005094402-0004/0 on worker-20261005091729-spark-worker-1-35533 (spark-worker-1:35533) with 2 core(s)
26/10/05 09:44:02 INFO StandaloneSchedulerBackend: Granted executor ID app-20261005094402-0004/0 on hostPort spark-worker-1:35533 with 2 core(s), 2.0 GiB RAM
26/10/05 09:44:02 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20261005094402-0004/1 on worker-20261005091729-spark-worker-2-35451 (spark-worker-2:35451) with 2 core(s)
26/10/05 09:44:02 INFO StandaloneSchedulerBackend: Granted executor ID app-20261005094402-0004/1 on hostPort spark-worker-2:35451 with 2 core(s), 2.0 GiB RAM
26/10/05 09:44:02 INFO NettyBlockTransferService: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 7080.
26/10/05 09:44:02 INFO NettyBlockTransferService: Server created on spark-driver 0.0.0.0:7080
26/10/05 09:44:02 INFO BlockManagerMaster: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
26/10/05 09:44:02 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, spark-driver, 7080, None)
26/10/05 09:44:02 INFO BlockManagerMasterEndpoint: Registering block manager spark-driver:7080 with 265.5 MiB RAM, BlockManagerId(driver, spark-driver, 7080, None)
26/10/05 09:44:02 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, spark-driver, 7080, None)
26/10/05 09:44:02 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, spark-driver, 7080, None)
26/10/05 09:44:02 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20261005094402-0004/0 is now RUNNING
26/10/05 09:44:02 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20261005094402-0004/1 is now RUNNING
26/10/05 09:44:03 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.0
26/10/05 09:44:04 INFO SparkContext: Added file /project/common.py at spark://spark-driver:7079/files/common.py with timestamp 1791193444074
26/10/05 09:44:04 INFO Utils: Copying /project/common.py to /tmp/spark-f179ae60-a633-421e-a83c-f76a6f57073/userFiles-cd1848e3-70ba-4632-8f5e-77d57d33cfc8/common.py
26/10/05 09:44:14 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
Exported 18,253,326 rows. Measurements: /project/artifacts/spark_6months_run2.json
Real runtime and resource measurements saved: /home/saiteja-ubuntu-31/taxi_a3/artifacts/spark_6months_run2.json
saiteja-ubuntu-31@saiteja-ubuntu-31-QEMU-Virtual-Machine:~/taxi_a3$ cd ~/taxi_a3
python3 -c 'py'
import json
r = json.load(open('artifacts/spark_6months_run2.json'))
print(f"spark run2: e2e={r['e2e_seconds']:.1f}s months={r['months']} rows_out={r['output']['rows']:,} "
      f"peak_cpu={r['resources']['cluster']['peak_cpu_percent']:.0f}% "
      f"peak_mem={r['resources']['cluster']['peak_memory_working_set_bytes']/2**30:.2f}GiB")
PY
spark run2: e2e=253.2s months=6 rows_out=18,253,326 peak_cpu=437% peak_mem=5.81GiB
saiteja-ubuntu-31@saiteja-ubuntu-31-QEMU-Virtual-Machine:~/taxi_a3$
```

Figure 9. Run 2 result: executors granted on both workers (2 cores, 2.0 GiB each), 18,253,326 rows exported, and the recorded time of 253.2 s with 437% peak CPU and 5.81 GiB peak memory.

```
saiteja-ubuntu-31@saiteja-ubuntu-31-QEMU-Virtual-Machine:~/taxi_a3$ sudo docker exec ray-head python3 verify_outputs.py \
  --spark /project/outputs/spark_check \
  --ray /project/outputs/ray_joinfixed_check \
  --result /project/artifacts/parity_joinfixed_check.json
Exact output parity: PASS
Spark rows: 2,884,428; Ray rows: 2,884,428; differences: 0/0
saiteja-ubuntu-31@saiteja-ubuntu-31-QEMU-Virtual-Machine:~/taxi_a3$ sudo docker exec ray-head python3 verify_outputs.py \
  --spark /project/outputs/spark_check \
  --ray /project/outputs/ray_joinfixed_check \
  --result /project/artifacts/parity_joinfixed_check.json
Exact output parity: PASS
Spark rows: 2,884,428; Ray rows: 2,884,428; differences: 0/0
saiteja-ubuntu-31@saiteja-ubuntu-31-QEMU-Virtual-Machine:~/taxi_a3$ sudo sync
echo 3 | sudo tee /proc/sys/vm/drop_caches
sudo python3 measure_run.py \
  --metrics artifacts/ray_joinfixed_6months_run1.json \
  --containers ray-head ray-worker-1 ray-worker-2 \
  --workers ray-worker-1 ray-worker-2 \
  --docker exec ray-head python3 ray_clean_joinfixed.py \
  --address ray-head:6379 \
  --months 6 --partitions 64 --shuffle pull \
  --read-blocks-per-file 1 --block-mib 64 \
  --output /project/outputs/ray_joinfixed_6months_run1 \
  --metrics /project/artifacts/ray_joinfixed_6months_run1.json
3
2026-10-05 08:03:29,610 INFO worker.py:1771 -- Connecting to existing Ray cluster at address: ray-head:6379...
2026-10-05 08:03:29,639 INFO worker.py:1942 -- Connected to Ray cluster. View the dashboard at http://172.20.0.2:8265
2026-10-05 08:03:31,460 INFO logging.py:293 -- Registered dataset logger for dataset dataset_7_0
2026-10-05 08:03:31,518 INFO streaming_executor.py:159 -- Starting execution of Dataset dataset_7_0. Full logs are in /tmp/ray/session_2026-10-05_07-57-50_518653_7/logs/ray-data
2026-10-05 08:03:31,519 INFO streaming_executor.py:168 -- Execution plan of Dataset dataset_7_0: InputDataBuffer[Input] -> TaskPoolMapOperator[ReadTaxiParquetAndClean] -> AllToAllOperator[Sort] -> TaskPoolMapOperator[MapBatches(deduplicate_bucket)], InputDataBuffer[Input] -> JoinOperator[Join(num_partitions=64)] -> TaskPoolMapOperator[MapBatche
s(add_features)-write]
2026-10-05 08:16:30,072 INFO streaming_executor.py:279 -- ✓ Dataset dataset_7_0 execution finished in 778.55 seconds
2026-10-05 08:16:30,431 INFO dataset.py:4869 -- Data sink Parquet finished. 18253326 rows and 4.80GB data written.
Exported 18,253,326 rows. Measurements: /project/artifacts/ray_joinfixed_6months_run1.json
Real runtime and resource measurements saved: /home/saiteja-ubuntu-31/taxi_a3/artifacts/ray_joinfixed_6months_run1.json
```

Figure 10. Ray side: one-month exact output parity (differences 0/0) and the six-month Ray run (command, 778.55 s Ray Data execution, 18,253,326 rows exported). From screenshots/5_ray_6month_run_log.png.